Predictive modeling project

Yaroslav Kudrevych

January-March 2026

Contents

1 Main model **2**

1.1 Extending feature set 2

1.2 Introducing Fourier harmonic features 5

1.3 Introducing RBF features 5

1.3.1 Using Gaussian(RBF) kernel 6

1.3.2 Hyperparameter selection 7

2 Residue correction **8**

2.1 Motivation 8

2.2 Choosing right residue correction model 8

2.3 GBM selection 8

2.4 Hybrid model performance 9

3 Final error correction **10**

3.1 Introducing Kalman filter 10

3.2 Final performance 11

Introduction

This report documents our approach to time series forecasting during the regime change period, namely September 2022 - October 2023. The code of the project and its detailed description can be found in the dedicated GitHub repository.

1 Main model

Our primary model is a linear `QuantileRegressor`. This choice is motivated by a high n/p ratio ($n \approx 3400$ and $p = 35$), where p represents the variables common to both the training and testing sets. We use extensive feature engineering to capture the maximum signal from the data. The resulting model has low variance but is limited by higher bias. This performance limitation directly motivates the evolution of our final architecture, where we introduce additional layers for residual correction to resolve the complex patterns that remain.

1.1 Extending feature set

Below is the plot for variable importance for the `QuantileRegressor` on the standardized data without any additional features:

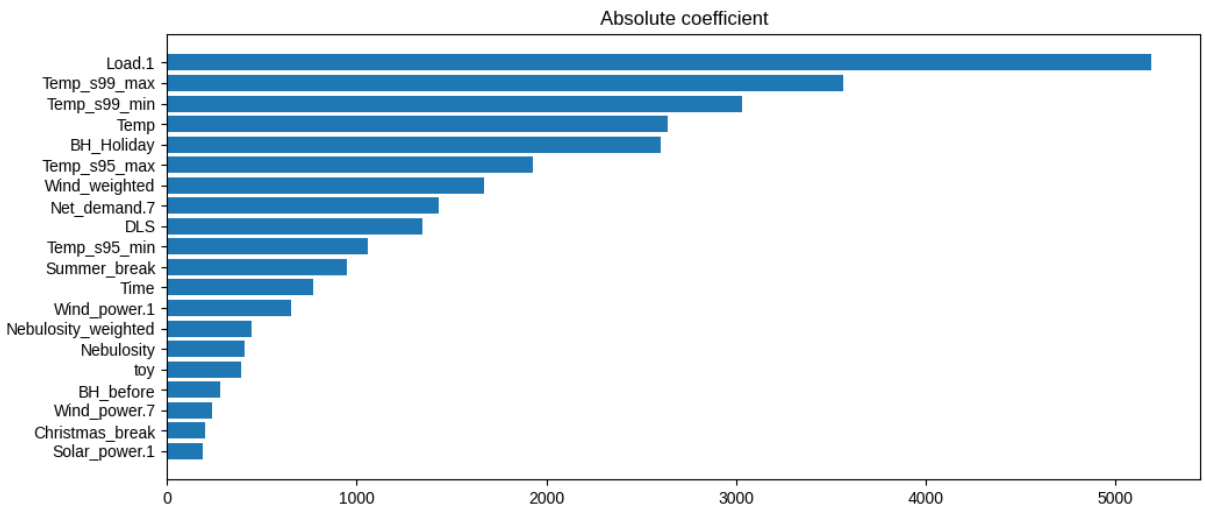


Figure 1: The most important initial features

Our initial feature set is clearly insufficient for our QuantileRegressor, as it has a priori low expressivity(variance). Thus, we are interested in adding more features to make our model more expressive. There are several directions for improvement:

- Add feature interaction
- Split some variables into categories
- Extract more information (e.g, a binary flag) from the dataset and use it as a new feature
- Perform a feature map on some variables (e.g., Polynomial, RBF, etc.)

In the rest of the chapter, we provide the features that we add to our linear model, along with justifications.

Temperature features. We observe that heat-driven electricity demand increases when the temperature deviates above or below a certain threshold, representing the limits of the thermal comfort zone. Hence, we introduce the following threshold-based features:

```
df['Heating_Std'] = np.maximum(288.15-df['Temp_s95'],0)
df['Cooling_Std'] = np.maximum(df['Temp_s95']-295.15,0)
```

Wind-related features. We would also like to start adding interactions between our features. We might start from `Wind_weighted` and `Heating_Std`. These variables are obviously correlated, as the combination of low (truncated) temperature and cold wind increases electricity consumption due to heat loss. We propose the following interaction:

```
df['Wind_Chill'] = df['Heating_Std'] * df['Wind_weighted']
```

The goal now is to continue adding variable interactions. Notably, we would like to see the dependence between such important variables as `Time` and `Wind_weighted`. This is justified as wind increases effective heat loss from buildings (the wind chill effect). The interaction between wind and temperature, therefore, affects heating-driven electricity demand.

```
df['Time_Cooling'] = df['Time'] * df['Wind_weighted']
```

Calendar features The next question is: how do we account for the calendar structure of net demand ? We answer this question by adding the following two features. One is simply the week number, which goes from 1 to 52. The second allows us to see the current day temperature compared to the weekly maximum. Indeed, it makes sense to compare those, as buildings tend to preserve heat or cold; thus, net demand after a warm day might be less than net demand after a cold one, with the same current temperature. On the other hand, keeping the week number lets our model memorize which time of year is the coldest.

```
df['Week']=df['Date'].dt.isocalendar().week.astype(int)
df['Temp_vs_Weekly_Max']=df['Temp']/df['Temp'].rolling(window=7,
    min_periods=1).max()
```

Another idea for improvement is adjusting our coeficients β with time. How can we do that ? We can try to make our time 'continuous', with respect of `Year`. Namely, we can discretize day by day

```
    df['Year_Continuous'] = df['Date'].dt.year + (df['Date'].dt.
        dayofyear / 366)
```

This feature is complementary to `toy` in the initial dataset. As we are also interested in forecasting the net demand before and after introducing energy sobriety, we might want to consider a checkpoint that would track the thermal sobriety regime. We might consider adding

```
inflection = (pd.to_datetime('2022-01-01') - self.train_date_min).
    days
df['Sobriety_Trend'] = np.maximum(0,df['Time']-inflection)
```

since there was a known structural change in 2022. Note that this is not feature leakage, as we chose to train our model on the data that includes 2022 and test it on data after 2022. Nevertheless, in practice, this feature adds at most 4-5 points to our performance.

Binary flags. We now introduce new qualitative variables. It was observed in the course that electricity consumption highly depends on the day of the week In particular, there is more consumption during working days, primarily due to industrial and commercial activity. Thus, we would like to distinguish between working days and the weekend. We add:

```
is_weekend = df['WeekDays'].isin([5, 6]).astype(int)
if 'Usage' in df.columns:
    df['work_activity'] = np.where((is_weekend == 0) & (df['Usage']
        == 'Public'), 1, 0)
else:
    df['work_activity'] = np.where(is_weekend == 0, 1, 0)
```

The same goes for the monthly consumption structure. It was observed that peak electricity consumption occurs in December and January. So, analogous to the weekend flag, we add:

```
df['Is_Peak_Month']=df['Date'].dt.month.isin([12,1,2]).astype(int)
```

In total, we added 10 new variables to our model. To prove that they are important, we calculate the Pearson correlation coefficient of each variable with the target and consider its associated p-value using the `pearsonr` function from the `scipy.stats` package.

This function tests the null hypothesis that the two variables are uncorrelated. We note that the p-value relies on the assumption of normality of the sampling distribution of the correlation coefficient and not of the data itself. With n>3000, this is essentially guaranteed by the Central Limit Theorem, regardless of the true underlying distribution. Below is the table of the `pearsonr` function outputs for each variable added:

Variable	Correlation coeff.	p-value
Sobriety_Trend	-0.19	2.8×10^{-30}
Heating_Std	0.89	0.0
Cooling_Std	-0.14	1.51×10^{-16}
Wind_Chill	0.77	0.0
Temp_vs_Weekly_Max	-0.13	1.75×10^{-14}
Week	-0.21	5.71×10^{-35}
Time_Cooling	-0.15	1.71×10^{-19}
Is_Peak_Month	0.66	0.0
Work_activity	0.27	3.58×10^{-61}
Year_Continuous	-0.18	2.714×10^{-27}

Table 1: Variable importance in linear model. We put 0 for the features whose p-values were lower than 10^{-308}.

1.2 Introducing Fourier harmonic features

It was proven during the course that adding periodic features can improve the performance of the model. Notably, adding periodic Fourier features can help substantially. We thus add `n_Fourier` pairs of the Fourier harmonic features:

```
w = 2 * np.pi / 365.25
n_fourier = self.n_Fourier_features
for i in range(1, n_fourier + 1):
    df[f'cos{i}'] = np.cos(df['Time'] * w * i)
    df[f'sin{i}'] = np.sin(df['Time'] * w * i)
```

1.3 Introducing RBF features

Currently, our linear model is not expressive enough to obtain good predictions. One of the ways to improve its performance is to add new features to the feature space. In particular, we may extend our linear model by adding a feature map. Owing to the *kernel trick*, which is well described in the literature, we may allow ourselves to choose only a kernel adapted for our purpose.

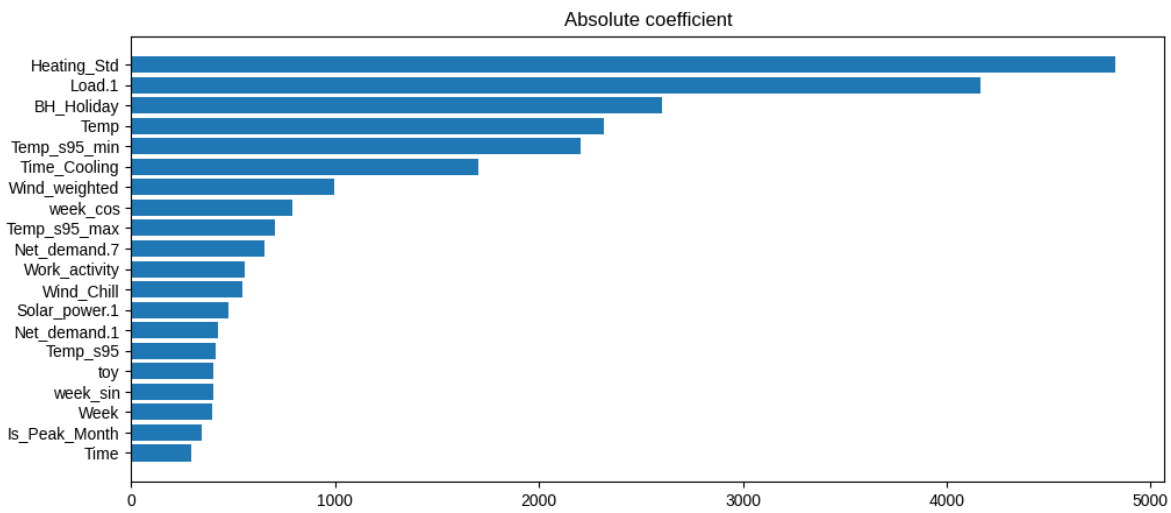


Figure 2: The most important features after initial feature engineering

We can observe that the most important features, except for the lagged ones, include `Heating_Std`, `Temp_s95_min`, `Time_cooling`, `Temp`, `Wind_weighted`, and `Wind_Chill`.

More precisely, out of the top 20, we have:

- 4 - lagged features
- 3 smoothed temperature features
- 4 categorical variables
- 9 quantitative variables

We would like to concentrate our attention on exploiting these 9 covariables more.

1.3.1 Using Gaussian(RBF) kernel

A radial kernel, otherwise known as a Gaussian kernel, is a popular way to increase feature space. It has the important property of *universality*, which allows it to approximate any smooth function. Moreover, its implicit feature space is infinite-dimensional, which gives additional motivation to use it.

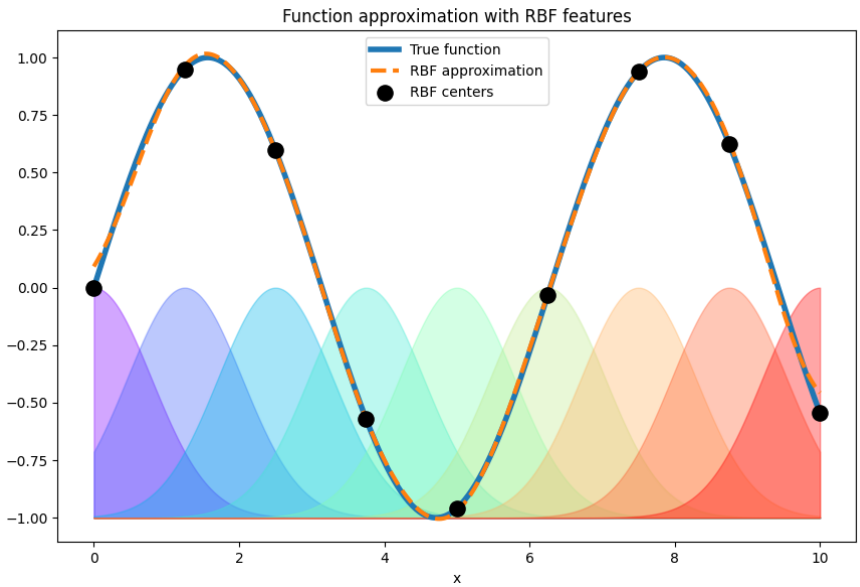
We recall that the RBF kernel is of the form:

$$K(x, y) = \exp\left(-\gamma \times \|x - y\|^2\right) \quad , \text{for } \gamma > 0$$

We know that the function $x \to e^{-x^2}$ is very concentrated around 0. Hence, the 'locality' of our RBF kernel. So, to model a smooth function well, we will need to create several RBF features. Thus, there are two parameters to consider: the number of RBF features for a certain variable and a width parameter $\sigma > 0$. For the sake of model interpretability, let us set σ equal for all the RBF features of one variable.

Temperature

Clearly, `Temp` is one of the most important features. As its smoothed temperature also has great importance, we have a strong interest in further exploiting the `Temp` feature. Simple analysis shows that the range of `Temp` over all the data is $[-4.022, 29.394]$ degrees Celsius. Thus, we are tempted to space our RBF features evenly (for simplicity) in the range $= [-5, 30]$ degrees Celsius. Yet, there is one caveat to it. Note the picture below:



It is visible that the 'tails' become slightly larger as they approach the ends of the considered interval. We would like to account for that. So instead, we will space our RBF features' anchors in the interval $[-10, 35]$ degrees Celsius. Let us put one 'anchor' every 5 degrees. We thus obtain 10 RBF features associated with `Temp`. Below is the code for these features. It will be similar for all 3 sets of RBF features that we introduce.

```
temp_min, temp_max = 273.15 - 10, 273.15 + 35
temp_anchors_k = np.linspace(temp_min, temp_max, 9).reshape(-1, 1)
temp_values = df['Temp'].values.reshape(-1, 1)
gamma_k = self.gamma_temp # choose by cross-validation
```

```
rbf_features = rbf_kernel(temp_values, temp_anchors_k, gamma=
    gamma_k)
for i in range(rbf_features.shape[1]):
    df[f'Temp_K_RBF_{i}'] = rbf_features[:, i]
```

Nebulosity We recall that the values of Nebulosity are in $[0, 1]$. We thus place three evenly spaced anchors as centers for our RBF features, namely $C_i = [0.0, 0.5, 1.0]$. `Nebulosity` has smaller range than `Temp` and are less important, so we can initialize our anchors at the borders (at min and max values).

Wind_chill The range of `Wind_Chill` is $[0, 114.4]$. This is a big range, but `Wind_Chill` is much less important than `Temp`. So we place our anchors exactly at the borders: $C_i = [0, 114.4]$ to account for eventual 'extremal' behavior of this feature.

1.3.2 Hyperparameter selection

For training, we use the following split: Train set: March 2013 - March 2022; Calibration set: April 2022 - May 2022; Validation set: June 2022-August 2022.

There is another approach to the train/validation split, which consists of using the data before 2020 (COVID) for training and the data after for calibration/validation. However, we would like to emphasize that these two splits lead to roughly the same validation score and almost identical Kaggle score (<2 points of difference) for the considered model.

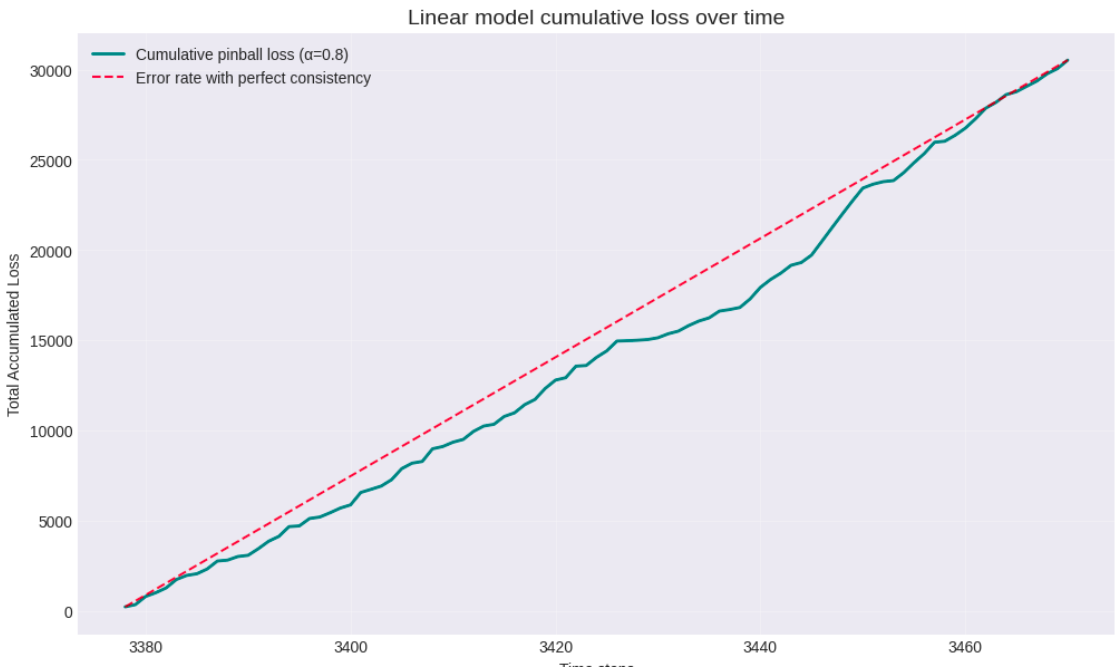
For our main model, we have 5 hyperparameters to tune: λ of L1 regularization, `n_Fourier` number of Fourier harmonics to add, and γ_{Temp}, γ_{Neb}, γ_{Wch} the widths of each RBF feature for 3 RBF feature sets. As we cannot use common cross-validation for time-series data, we use the 'growing window' approach implemented in `TimeSeriesSplit` [1] class from `scikit-learn` library. We also use `pipeline` from the same library to standardize the process of hyperparameter selection.

We first want to choose the scale of our hyperparameters. This constitutes our first `GridSearchCV`. Then, when the scale is known, we want to perform a more precise grid search to choose the final values for our hyperparameters. We chose a grid of 288 total values for 5 hyperparameters. With 10-fold `TimeSeriesSplit`, this results in $10 \times 288 = 2880$ fits on the training data. It takes from 5 to 7 minutes to run this search. The code for this search can be found in the file `cv_rbf.py`.

2 Residue correction

2.1 Motivation

Consider the error consistency analysis of our linear model:



We can observe that our first model predictions are not totally consistent, i.e., they don't grow in a steady manner. This motivates the addition of another layer to our model, which would try to learn the residues of the first and thus a pattern beyond the inconsistency of the first model residues.

2.2 Choosing right residue correction model

As a second layer of our model, we would like to predict the residues of our first model, which is aimed at improving the prediction. Ideally, we should pick a model complementary to the linear model described above. That is, we would want a non-linear model. Moreover, we have created many features based on smooth functions, so we would like a model that, on the contrary, deals with 'rupture'-like dependencies between variables. Given these parameters, we can conclude that a good residue correction model would be a gradient boosted tree.

2.3 GBM selection

We intend to create a model that will serve as a 'helper' for the main linear model. Hence, we constrain the architecture of our GBM so that it does not become too complex. Notably, our GBM should be especially aimed at combating overfitting.

The main parameters of GBM are as follows:

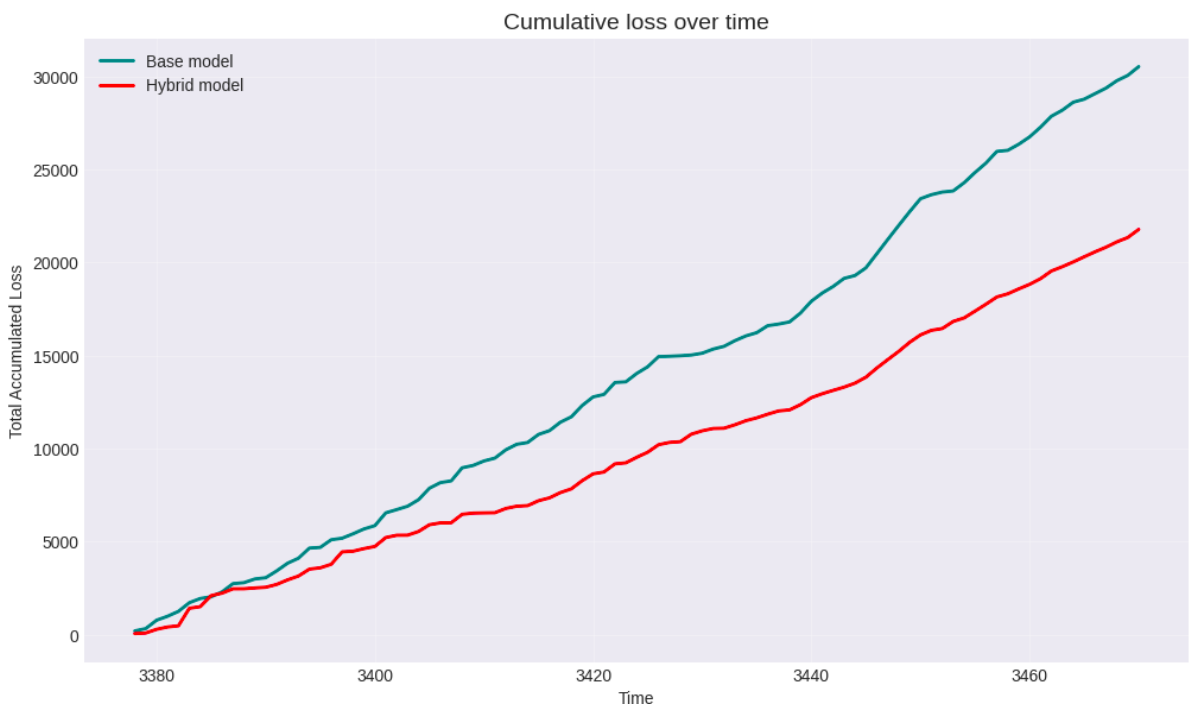
- Tree depth. We want it to be as small as possible, in the range of 4-8. We also set the number of leaves equal to $2^{depth} - 1$.
- Minimal leaf size. This parameter also controls overfitting, so we have to choose it carefully.
- Target quantile. It has been observed that GBMs tend to systematically over-predict the target quantile, so we have to choose a value < 0.8 that gives the best result.
- Regularization parameter. Here, we use only L2 regularization for simplicity. It naturally controls overfitting as well.

This naturally leads us to model selection. How can we perform it ? We decided to choose the model that performs best on the validation score, as training a GBM is more time consuming than training a linear model. We set the grid of 4 parameters, with a total of 192 hyperparameter combinations. We decided to use `lightgbm.Booster` [2] for our residue correction model.

The code of this grid search is available in the file `select_gbm.py`.

2.4 Hybrid model performance

We would like to evaluate our model performance after the addition of the residue correction layer. To do this, we consider the cumulative residues of the linear and hybrid models:



We can see on this plot that residue correction by GBM significantly improved performance. More precisely, it improved our validation score by almost 30%.

3 Final error correction

3.1 Introducing Kalman filter

The last layer of our model is error correction. There are several ways to do it. The most common include ARIMA and the Kalman filter. There is also a more powerful VIKING [3] model, but we wouldn't be able to explain its use properly. Here, we prefer to use the Kalman filter.

Since we observe a single measurement—the `Net_Demand` —the standard matrix equations simplify significantly. Namely, we define our system by two equations:

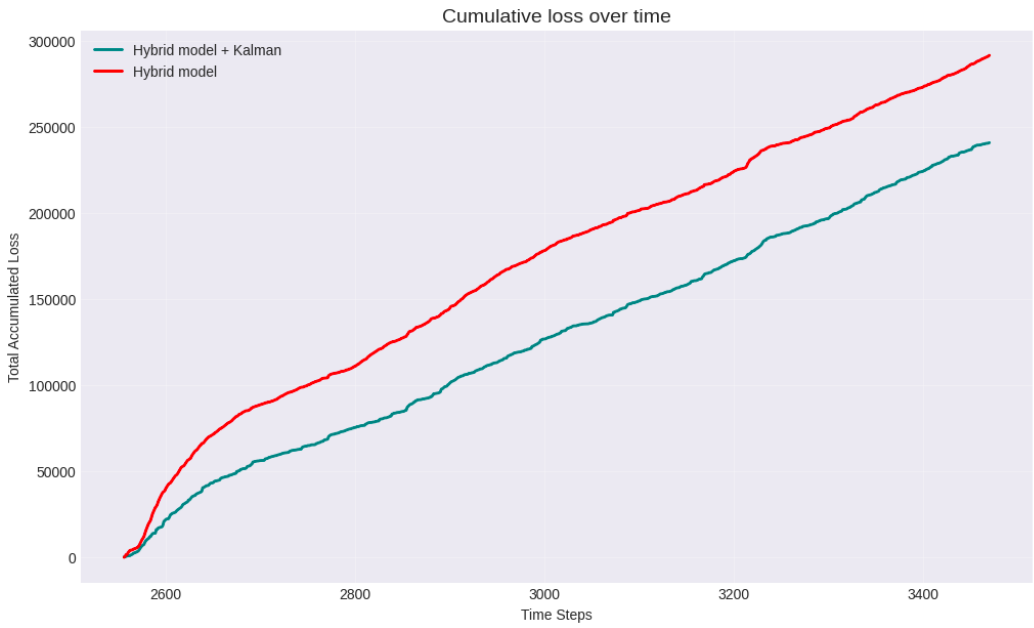
- State transition:
$$x_t = x_{t-1} + w_t, \quad w_t \sim N(0, Q)$$
where Q (model uncertainty) represents how quickly the grid's behavior drifts, and
- Observation equation: v_t.
$$z_t = x_t + v_t, \quad v_t \sim N(0, R)$$
Where R (measurement uncertainty) represents the inherent volatility of the demand signal, z_t is the observed residual:$z_t = y_{\text{true}} - (y_{\text{pred_base}} + B)$ and B is the static bias.

Our Kalman filter implementation is as follows:

- Initialize $R = Q = 1$, $P_0 = 1$, $B =$ mean of residuals over the training data, and $\hat{x}_0 = 0$.
- At each time step t, perform:
  1. Covariance Prediction: $P_{t|t-1} = P_{t-1|t-1} + Q$. We project our uncertainty P forward. Since we assume the bias is a random walk, the state prediction is simply the previous state.
  2. Innovation: $\tilde{y}_t = y_{\text{true},t} - (y_{\text{pred_base},t} + B + \hat{x}_{t-1})$
  3. Kalman Gain: $K_t = \frac{P_{t|t-1}}{P_{t|t-1}+R}$
  4. We want to penalize over-predictions less than under-predictions. Thus, we introduce the update weight: $\omega_t = \begin{cases} \tau & \text{if } \tilde{y}_t > 0 \\ 1-\tau & \text{if } \tilde{y}_t \leq 0 \end{cases}$
  5. State Update:
  $$\hat{x}_t = \hat{x}_{t-1} + K_t \cdot \tilde{y}_t \cdot \omega_t$$
  6. Covariance Update:
  $$P_t = (1 - K_t)P_{t|t-1}$$
- Our final adjusted forecast is then:
$$\hat{y}_{\text{final},t} = y_{\text{pred_base},t} + B + \hat{x}_t$$
where B is the static bias, and $\hat{x}_t$ is the dynamic bias.

3.2 Final performance

It was observed that the validation set containing June-August 2022 is too short to show the filter performance. Thus, we present the performance of the filter with the same model, but with the training set up to 2020 and the validation set from March 2020 to August 2022. This performance is reproducible and may be seen in the file `kalman_interactive.ipynb`. The final model placed in `model_submission.py` gives a Kaggle score of 279.49.



References

[1] Scikit-learn contributors. *sklearn.model_selection.TimeSeriesSplit documentation.* Available: `https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html`.

[2] Microsoft Corporation. *LightGBM Python API: lightgbm.Booster documentation.* Available: `https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.Booster.html`.

[3] J. de Vilmarest, Y. Goude, and O. Wintenberger. *Viking: Variational Bayesian Variance Tracking Winning a Post-COVID Day-Ahead Electricity Load Forecasting Competition.* [Online]. Available: `https://roseyu.com/time-series-workshop/submissions/2021/TSW-ICML2021_paper_15.pdf`.